МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ

РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего образования

«Московский авиационный институт

(национальный исследовательский университет)»

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 2

Дисциплина:

Операционные системы и архитектура компьютера

Тема:

Управление процессами и обмен данными посредством каналов

Выполнил:

студент группы М8О-201БВ-25

Пантюхин Никита Олегович

Проверил:

Преподаватель

Миронов Евгений Сергеевич

Москва, 2026 г.

Содержание

1 Цель работы **3**

2 Краткое описание архитектуры решения **3**

2.1 Схема взаимодействия компонентов 3

3 Листинг ключевых фрагментов кода **4**

4 Результаты работы **7**

4.1 Консольный вывод или логи 7

4.2 Скриншоты результатов 9

5 Выводы **10**

1. Цель работы

Цель работы — на практике изучить взаимодействие процессов в Linux: создание дочернего процесса с помощью `fork()`, передачу данных через неименованный канал `pipe`, перенаправление стандартного ввода функцией `dup2()`, запуск отдельной программы через `exec` и ожидание её завершения с помощью `waitpid()`.

2. Краткое описание архитектуры решения

Лабораторная работа состоит из двух отдельных программ: `parent` и `child`. Программа `parent` запрашивает имя выходного файла, создаёт один неименованный канал и вызывает `fork()`. В дочерней ветке закрывается write-end канала, а read-end перенаправляется в `STDIN_FILENO` с помощью `dup2()`. После этого выполняется `execl("./child ...)`, который заменяет программу дочернего процесса программой `child`.

Родительский процесс закрывает read-end канала, читает строки чисел с клавиатуры и передаёт их через write-end. Программа `child` получает эти строки через обычный `stdin`, проверяет целые числа, вычисляет сумму каждой строки и записывает результат в выбранный файл. После EOF родитель закрывает канал и ожидает завершения дочернего процесса через `waitpid()`.

Программы запускаются из каталога `build/`, поэтому файлы `parent` и `child` находятся рядом. Сигнал `SIGPIPE` игнорируется простым вызовом `signal()`, чтобы при преждевременном завершении `child` родитель мог получить и обработать ошибку записи в канал.

2.1. Схема взаимодействия компонентов

На рисунке показано направление передачи данных между пользователем, родительским процессом, каналом, дочерним процессом и выходным файлом.

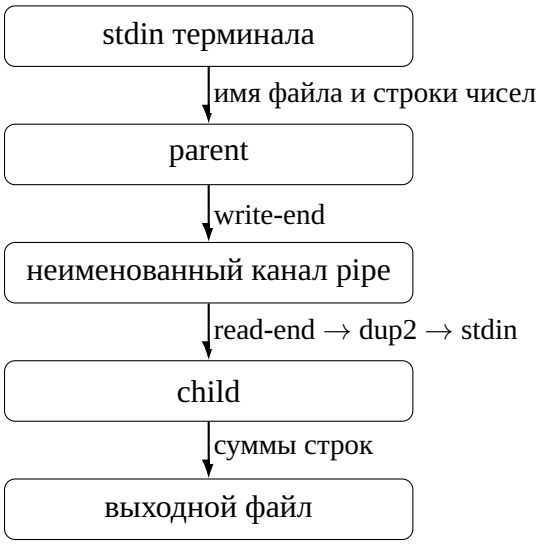


Рисунок — Схема взаимодействия компонентов программы

Канал создаётся до `fork()`, поэтому оба процесса получают его файловые дескрипторы. Каждый процесс закрывает неиспользуемый конец канала. Благодаря `dup2()` программа `child` может читать данные через стандартный ввод и не зависит от деталей передачи данных в `parent`.

3. Листинг ключевых фрагментов кода

Сначала родитель создаёт канал и дочерний процесс. При ошибках системных вызовов программа выводит сообщение и завершает работу.

```
int pipefd[2];

if (pipe(pipefd) == -1) {
    perror("pipe");
    /* освобождение ресурсов */
    return 1;
}

pid_t pid = fork();

if (pid == -1) {
    perror("fork");
    /* освобождение ресурсов */
    return 1;
}
```

Листинг 1 — Создание канала и дочернего процесса

В дочерней ветке write-end закрывается, read-end становится стандартным вводом, после чего запускается отдельная программа `child`.

```
if (pid == 0) {
    close(pipefd[1]);

    if (dup2(pipefd[0], STDIN_FILENO) == -1) {
        perror("dup2");
        close(pipefd[0]);
        free(filename);
        _exit(1);
    }

    close(pipefd[0]);
    execl("./child", "child", filename, NULL);
    perror("execl");
    free(filename);
    _exit(1);
}
```

Листинг 2 — Перенаправление stdin и запуск программы child

Родитель читает строки до EOF и передаёт каждую строку целиком через канал. Функция `write_all()` учитывает возможность частичной записи.

```
while ((line_length = getline(&line, &line_size, stdin)) != -1) {
    if (write_all(pipefd[1], line, (size_t)line_length) == -1) {
        parent_error = 1;
        break;
    }
}

free(line);
close(pipefd[1]);
```

Листинг 3 — Передача пользовательских строк через pipe

Программа `child` читает строку из перенаправленного `stdin`, разбирает её на целые числа и записывает вычисленную сумму в файл.

```
while (getline(&line, &line_size, stdin) != -1) {
    long long sum = 0;
    char *token = strtok(line, " \t\r\n");
    while (token != NULL) {
        errno = 0;
        char *end = NULL;
        long number = strtol(token, &end, 10);
        if (end == token || *end != '\0' || errno == ERANGE ||
            number < INT_MIN || number > INT_MAX) {
            fprintf(stderr, "Invalid number: %s\n", token);
            return cleanup_on_error(line, file_fd);
        }
        sum += number;
        token = strtok(NULL, " \t\r\n");
    }
    char result[64];
    int result_length = snprintf(result, sizeof(result), "%lld\n",
   sum);
    if (write_all(file_fd, result, (size_t)result_length) == -1) {
        return cleanup_on_error(line, file_fd);
    }
}
```

Листинг 4 — Чтение чисел, вычисление суммы и запись результата

После закрытия канала родитель ожидает дочерний процесс и проверяет его статус. Это не позволяет завершившемуся дочернему процессу остаться в состоянии zombie.

```
int status;
if (waitpid(pid, &status, 0) == -1) {
    perror("waitpid");
    return 1;
}
if (check_child_status(status) == -1) {
    parent_error = 1;
}
```

Листинг 5 — Ожидание и проверка завершения child

Полный исходный код доступен в GitHub-репозитории лабораторной работы № 2.

4. Результаты работы

Проект был собран с помощью CMake без ошибок и предупреждений. В успешном сценарии программа получила несколько строк целых чисел и записала сумму каждой строки в файл `result_file.txt`.

Введённая строка	Результат
2 5 6 7 10 3	33
3 9 93939 930 3434	98315
1 22 3	26
453 34 35 78	600
34 7 6 9 53	109

Таблица — Результаты вычисления сумм

После завершения ввода через EOF родитель закрыл write-end канала, дочерний процесс закончил чтение и корректно завершился. Содержимое выходного файла совпало с рассчитанными суммами.

4.1. Консольный вывод или логи

В консоли выводятся PID родительского и дочернего процессов. Это позволяет увидеть, что `parent` и `child` работают как два разных процесса.

```
$ ./parent
Enter filename: result_file.txt
Parent PID: 317763, Child PID: 318229
Enter numbers line by line. Press Ctrl+D to finish.
Child PID: 318229, Parent PID: 317763
2 5 6 7 10 3
43 8598 98 9485 9459 89485 94898 5948 98984
3 9 93939 930 3434
1 22 3
453 34 35 78
34 7 6 9 53
^D
```

Листинг 6 — Пример успешного запуска программы

После завершения программы файл содержит по одной сумме для каждой введённой строки.

```
$ cat result_file.txt
33
316998
98315
26
600
109
```

Листинг 7 — Проверка содержимого выходного файла

Для проверки системных вызовов программа запускалась из каталога `build/` под управлением `strace`. В отчёт включён только фрагмент, относящийся к взаимодействию процессов.

```
322069 execve("./parent", ["./parent"], ...) = 0
322069 pipe2([3, 4], 0) = 0
322069 clone(..., SIGCHLD, ...) = 322296
322069 close(3) = 0
322296 close(4) = 0
322296 dup2(3, 0) = 0
322296 close(3) = 0
322296 execve("./child", ["child", "result_file.txt"], ...) = 0
322069 write(4, "2 5 6 7 10 3\n", 17) = 17
322296 write(3, "33\n", 3) = 3
322069 close(4) = 0
322069 wait4(322296, ..., 0, NULL) = 322296
```

Листинг 8 — Ключевые системные вызовы из трассировки strace

Трассировка показывает создание канала и второго процесса, перенаправление файлового дескриптора, запуск программы `child`, передачу строки через канал и ожидание завершения. В Linux библиотечным вызовам `pipe()`, `fork()`, `execl()` и `waitpid()` в данном запуске соответствуют системные вызовы `pipe2()`, `clone()`, `execve()` и `wait4()`.

4.2. Скриншоты результатов

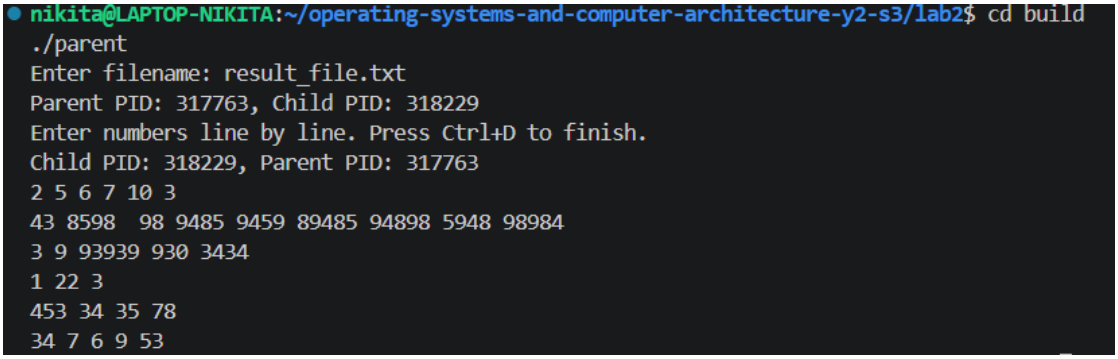


Рисунок — Успешный запуск родительского и дочернего процессов

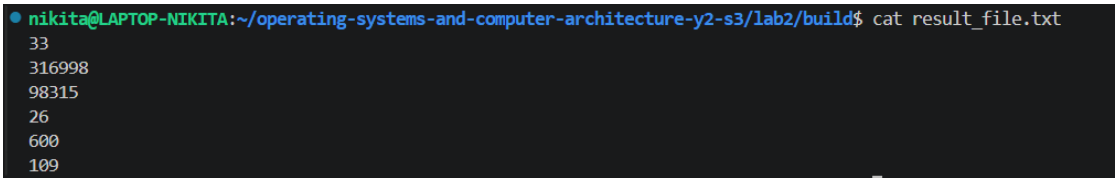


Рисунок — Результаты вычисления сумм

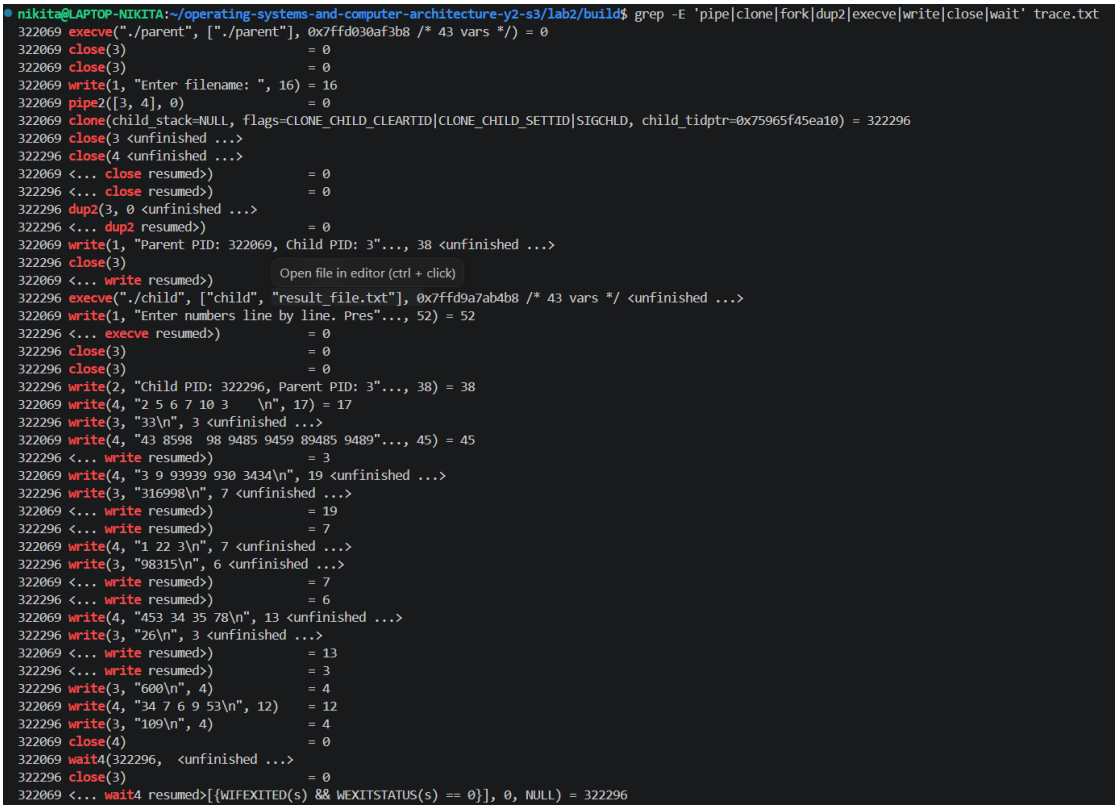


Рисунок — Системные вызовы при взаимодействии процессов

5. Выводы

После выполнения этой лабораторной я лучше понял, как в Linux взаимодействуют родительский и дочерний процессы и как данные передаются между ними через `pipe`. На практике я разобрался, как вместе используются `fork()`, `dup2()`, `exec()` и `waitpid()`, а также зачем закрывать неиспользуемые концы канала. С помощью `strace` я увидел системные вызовы, которые реально выполняются при работе программы. Благодаря этому тема процессов и межпроцессного взаимодействия стала для меня намного понятнее.